



UNIVERSITY
OF COLOGNE

WORKSHOP 09

Advanced Analytics and Applications

Agenda

1

Lecture Recap

2

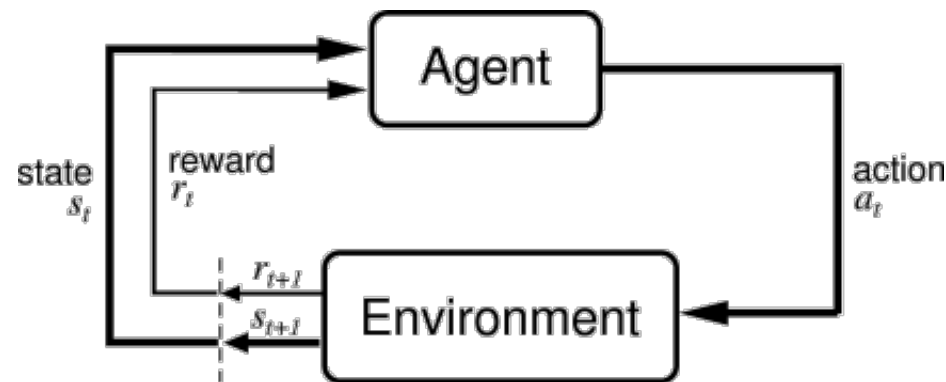
Exemplary Exam Questions

3

Coding Tasks

Motivation

- Contrary to supervised learning before, what is the goal when we have a **decision problem**?
- Consider this:
 - An **agent** can interact with an *environment* through carrying out an **action**.
 - It observes a special temporary view of it, the **state**, and as a result of its actions a **reward**.
 - What is the best possible course of action in this environment?



Markov Decision Process

- We formalize RL environments as **Markov Decision Processes** (MDP):
 - Set of states $\mathcal{S}$, set of actions $\mathcal{A}$
 - State transition probability function $P(s'|s, a)$, Reward function $R(s, a, s')$
 - Discount factor (importance of not-immediate future) $\gamma \in [0,1]$
- **Goal**: Find policy $\pi(a|s)$ that maximizes expected return.
- First step: Formalize expected **return**
 - $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ (non-episodic task, generalizes to episodic)
- Second step: Calculate **value** of being in a state, executing an action
 - $v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s]$ (state-value)
 - $q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a]$ (state-action value)

Bellman Equations

- Now that we know the value of being in a state / taking an action in a state (previous slide, called **Bellman Expectation Equation**), we can decide how to behave:
- The **Bellman Optimality Equations** are
 - The value of a state under an optimal policy is the expected return from the best possible action in that state:

$$V_*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_*(s')]$$

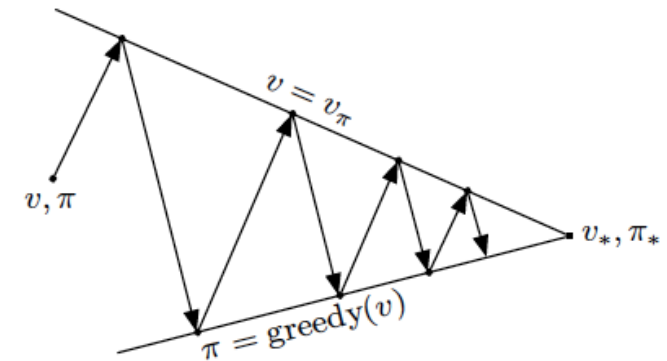
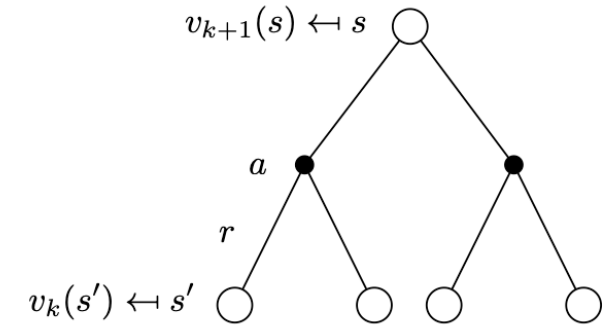
- The value to take an action under an optimal policy is given by:

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$$

- We consider an MDP *solved* if we know these (because then we can derive an optimal policy!)

Solving MDP with a Model

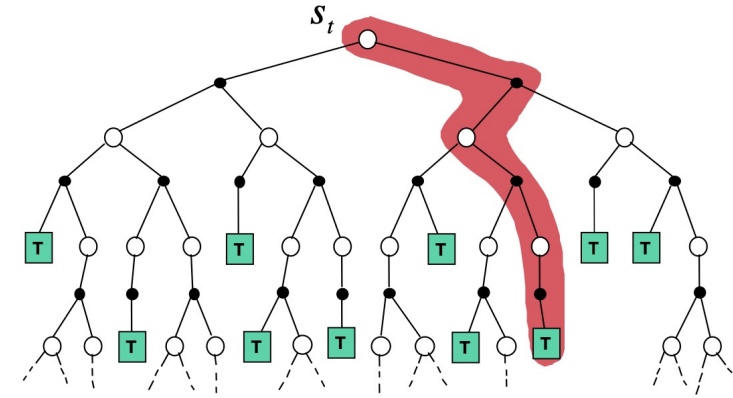
- If we have **model** of the environment including state transition probabilities, we can exploit that and *plan* the optimal policy using **Dynamic Programming**
- The problem consists of two problems actually:
 - **Prediction Problem:** Estimating the value functions given a policy
Compute values iteratively using the Bellman equations
 - **Control Problem:** Finding optimal policy (and therefore optimal value functions). Improve the policy by acting greedily w.r.t. the newly estimated values
- These two together make up **policy iteration**.
- **Value iteration** as an alternative combines that into one update step that uses the fact that we know that the value of the state will be determined by the best possible action we can take (given our current best knowledge)



Solving MDP without a Model

- As soon as we have no model of the environment, we are doing Reinforcement Learning!
- We have to learn purely from experience in the environment!
- There are different types of algorithms with regard to
 - What they use to learn (full episodes vs. single step)
 - What they learn (their own policy or an expert policy)
 - How they learn (tabular value lookup or function approximation)
 - What they can do (continuous or discrete action space)
 - ...

Monte Carlo



- Without the model, we can't predict the outcome of actions to make claims about the value of a state. We need to learn the **value of an action** in a state!
- Monte Carlo estimates action values $Q(s, a)$ for taking action a in state s by **experiencing a full episode** (until terminal state T) and averaging the returns using an update rule such as:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[G_t - Q(s_t, a_t)]$$

Returns are the cumulative rewards, discounted:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$$

- How do we experience? What is our policy π ?
- To ensure we explore the environment we use an **ϵ -greedy policy** where we pick the greedy action (highest currently known Q -value) with probability $1 - \epsilon$ and a random action for exploration with probability ϵ

Agenda

1

Lecture Recap

2

Exemplary Exam Questions

3

Coding Tasks

Multiple Choice Questions

What is the goal with episodic decision processes?

- a) Maximize immediate reward
- b) Minimize steps to reach terminal state
- c) Maximize episode return

Multiple Choice Questions

What is the goal with episodic decision processes?

- a) Maximize immediate reward
- b) Minimize steps to reach terminal state
- c) **Maximize episode return**

Multiple Choice Questions

How would you solve the MDP of playing chess?

- a) Monte Carlo (model-free, complete episodes available)
- b) Dynamic Programming (model-based)
- c) Temporal-Difference (model-free, online)

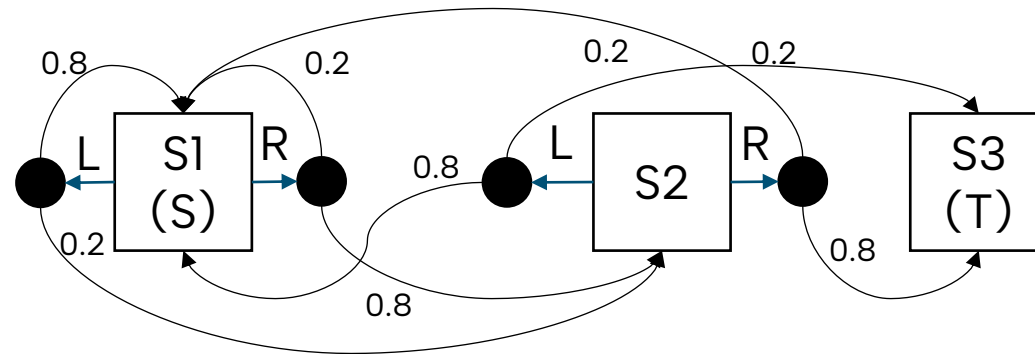
Multiple Choice Questions

How would you solve the MDP of playing chess?

- a) **Monte Carlo (model-free, complete episodes available)**
- b) Dynamic Programming (model-based)
- c) Temporal-Difference (model-free, online)

Dynamic Programming

Consider the following simple grid world with start (S) and terminal state (T). Possible actions are Left (L) and Right (R), but every step has a certain chance (20%) to slip in the opposite direction. Bumping into the boundary keeps the agent in the same state. Rewards are -1 for every step or +10 for reaching the terminal state (T).



- **Question:** Solve this MDP with Value Iteration (2 steps are enough). Initial values are $V_0(s) = 0 \quad \forall s \in S$

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

Dynamic Programming

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

Reward: -1 for each step, +10 for reaching (T)

Answer:

- First iteration (k=1):

To find the maximizing action ($\max_a$) we need to calculate the expectation sum for all a first:

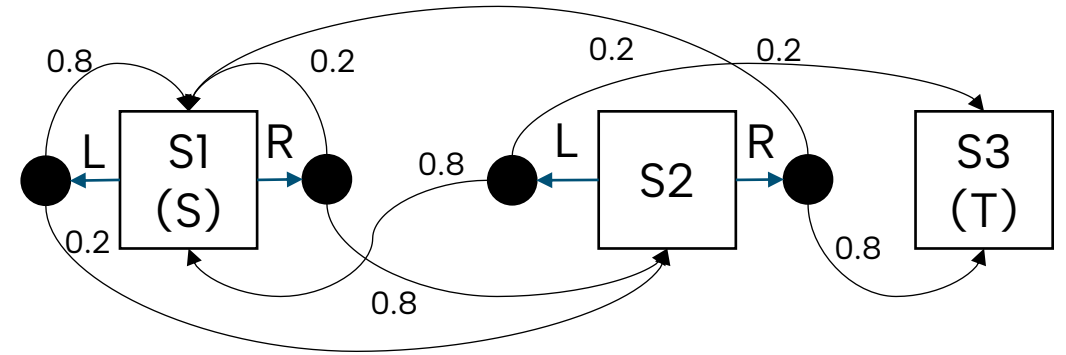
$$\sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

- For S1:

- Action L: $0.8 * [-1 + V_0(S2)] + 0.2 * [-1 + V_0(S1)] = 0.8 * [-1 + 0] + 0.2 * [-1 + 0] = -1$
- Action R: $0.8 * [-1 + V_0(S1)] + 0.2 * [-1 + V_0(S2)] = 0.8 * [-1 + 0] + 0.2 * [-1 + 0] = -1$
- Update $V_1(S1) = \max(-1, -1) = -1$

- For S2:

- Action L: $0.8 \cdot [-1 + V_0(S1)] + 0.2 \cdot [+10 + V_0(S3)] = 0.8 \cdot [-1 + 0] + 0.2 \cdot [10 + 0] = -0.8 + 2.0 = 1.2$
- Action R: $0.8 \cdot [+10 + V_0(S3)] + 0.2 \cdot [-1 + V_0(S1)] = 0.8 \cdot [10 + 0] + 0.2 \cdot [-1 + 0] = 8.0 - 0.2 = 7.8$
- Update $V_1(S2) = \max(1.2, 7.8) = 7.8$



Dynamic Programming

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

$$V_1(S1) = -1, V_1(S2) = 7.8$$

- Second iteration (k=2):

- For S1:

- Action L:

$$0.8 \cdot [-1 + V_1(S1)] + 0.2 \cdot [-1 + V_1(S2)] = 0.8 \cdot [-1 - 1.0] + 0.2 \cdot [-1 + 7.8] = -1.6 + 1.36 = -0.24$$

- Action R:

$$0.8 \cdot [-1 + V_1(S2)] + 0.2 \cdot [-1 + V_1(S1)] = 0.8 \cdot [-1 + 7.8] + 0.2 \cdot [-1 - 1.0] = 5.44 - 0.4 = 5.04$$

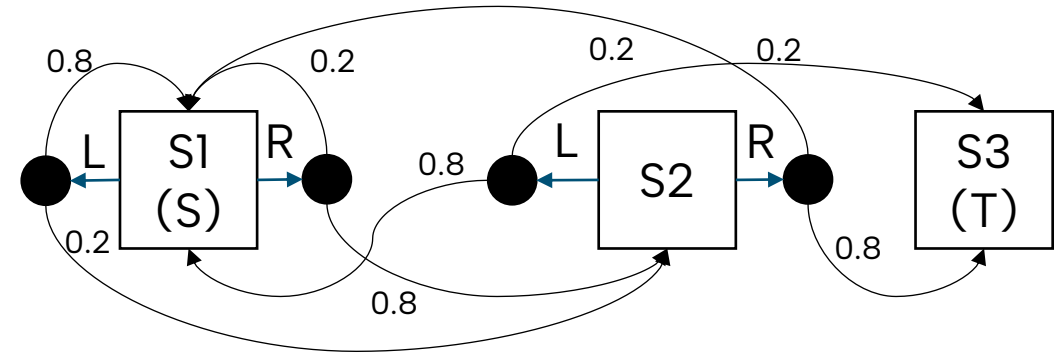
- Update $V_2(S1) = \max(-0.24, 5.04) = 5.04$

- For S2:

- Action L: $0.8 \cdot [-1 + V_1(S1)] + 0.2 \cdot [+10 + 0] = 0.8 \cdot [-1 - 1.0] + 2.0 = -1.6 + 2.0 = 0.4$

- Action R: $0.8 \cdot [+10 + 0] + 0.2 \cdot [-1 + V_1(S1)] = 8.0 + 0.2 \cdot [-1 - 1.0] = 8.0 - 0.4 = 7.6$

- Update $V_2(S2) = \max(0.4, 7.6) = 7.6$



Monte Carlo

First-visit MC only considers the first visit of each state
Every-visit MC considers multiple visits to each state

Question:

a) Consider an agent using **First-Visit Monte Carlo (MC)** to estimate the state-value function $V_{\pi}(s)$.

In an episode, state s_a is visited at $t = 5$ and $t = 10$.

The total discounted return from $t=5$ is $G_5 = 25$, and from $t=10$ is $G_{10} = 15$.

If the current estimate is $V(s_a) = 20$ and the learning rate $\alpha = 0.05$, what is the updated $V(s_a)$ after this episode?

b) What is a primary advantage of MC methods over Dynamic Programming?

Monte Carlo

Question:

a) Consider an agent using **First-Visit Monte Carlo (MC)** to estimate the state-value function $V_{\pi}(s)$.

In an episode, state s_a is visited at $t = 5$ and $t = 10$.

The total discounted return from $t=5$ is $G_5 = 25$, and from $t=10$ is $G_{10} = 15$.

If the current estimate is $V(s_a) = 20$ and the learning rate $\alpha = 0.05$, what is the updated $V(s_a)$ after this episode?

Answer: The update rule is $V(s_a) \leftarrow V(s_a) + \alpha(G_5 - V(s_a))$ as only the first visit at $t = 5$ is relevant.

Calculation:

$$\begin{aligned} V(s_a) &\leftarrow 20 + 0.05(25 - 20) \\ V(s_a) &= 20.25 \end{aligned}$$

Monte Carlo

Question:

b) What is a primary advantage of MC methods over Dynamic Programming?

Answer: The primary advantage of MC methods is that they are model-free, i.e., they do not need a model (transition probabilities and reward function) of the environment.

Agenda

1

Lecture Recap

2

Exemplary Exam Questions

3

Coding Tasks

Contact



For general questions and enquiries on **research, teaching, job openings** and **new projects** refer to our website at www.is3.uni-koeln.de



For specific **enquiries regarding this course** contact us by sending an email to the **IS3 teaching address** at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>